Lab 7 — Large Language Models (LLMs)

The Three Main Transformer Model Families

Family	Example Models	Main Use
Encoder-only	BERT, RoBERTa	Classification, sentiment analysis, NER
Encoder–decoder	T5, BART	Translation, summarization, question answering
Decoder-only	GPT family	Text generation, conversation, completion

1. Setup

```
!pip -q install transformers sentencepiece torch
```

```
from transformers import pipeline, set_seed

print("Libraries loaded successfully!")
```

```
Libraries loaded successfully!
```

Part A — Decoder-Only Model

Decoder-only example: GPT-style Text Generation

A decoder-only model predicts the next token based on previous tokens.

We will use **DistilGPT-2**, a small GPT-style model.

```
generator = pipeline("text-generation", model="distilgpt2")

prompt = "Natural language processing is"

output = generator(prompt, max_new_tokens=30, do_sample=False)

print(output[0]["generated_text"])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config.json: 100%  762/762 [00:00<00:00, 55.8kB/s]
model.safetensors: reconstructing file: 100%  353MB / 353MB, 28.0MB/s
model.safetensors: downloading bytes:  312MB, 23.8MB/s
Loading weights: 100%  76/76 [00:00<00:00, 1433.35it/s]
generation_config.json: 100%  124/124 [00:00<00:00, 2.03kB/s]
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 1.06kB/s]
vocab.json: 100%  1.04M/1.04M [00:00<00:00, 4.20MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 4.25MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 2.68MB/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'do_sample', 'max_new_tokens'}) is depre
[transformers] Both `max_new_tokens` (=30) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. P
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-proces
Natural language processing is a common feature of the language processing process.
```

Part B — Encoder–Decoder Model

Encoder–decoder example: Instruction / Text-to-Text Generation

Encoder–decoder models contain two main parts:

1. The **encoder** reads the input.
2. The **decoder** produces the output.

We will use **FLAN-T5-small**.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

prompt = "Translate to French: I love natural language processing."

# Convert text into tokens
output = model.generate(**tokenizer(prompt, return_tensors="pt"))

# Convert tokens back to text
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

```
config.json: 100%  1.40k/1.40k [00:00<00:00, 13.8kB/s]
tokenizer_config.json: 100%  2.54k/2.54k [00:00<00:00, 47.6kB/s]
spiece.model: reconstructing file: 100%  792kB / 792kB, 76.7kB/s
spiece.model: downloading bytes:  649kB, 62.8kB/s
tokenizer.json: 100%  2.42M/2.42M [00:00<00:00, 12.3MB/s]
special_tokens_map.json: 100%  2.20k/2.20k [00:00<00:00, 12.8kB/s]
model.safetensors: reconstructing file: 100%  308MB / 308MB, 25.3MB/s
model.safetensors: downloading bytes:  291MB, 21.7MB/s
Loading weights: 100%  190/190 [00:00<00:00, 3278.24it/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are
generation_config.json: 100%  147/147 [00:00<00:00, 2.79kB/s]
/usr/local/lib/python3.13/dist-packages/transformers/generation/utils.py:1954: UserWarning: Using the model-agnostic default `ma
  warnings.warn(
Je s'agissant de la technologie de linguage naturelle.
```

Part C — Encoder-Only Model

Encoder-only example: Sentiment Classification

Encoder-only models read the full input and create a representation of it.

They are very useful when the task is to **understand or classify text**.

Here we use a DistilBERT sentiment model.

```
classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")

text = "This NLP lab is easy and interesting."

result = classifier(text)
print(result)
```

```
config.json: 100%  629/629 [00:00<00:00, 10.4kB/s]
model.safetensors: reconstructing file: 100%  268MB / 268MB, 23.0MB/s
model.safetensors: downloading bytes:  250MB, 20.4MB/s
Loading weights: 100%  104/104 [00:00<00:00, 1762.54it/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 796B/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 890kB/s]
[{'label': 'POSITIVE', 'score': 0.999778687953949}]
```

Tasks

Complete the following tasks.

Task 1 — Model Families

For each task below, choose the most suitable model family:

1. Sentiment classification
2. Machine translation
3. Open-ended text generation
4. Named Entity Recognition
5. Text summarization

```
answers = {
    "Sentiment classification": "Encoder-only (BERT, DistilBERT)",
    "Machine translation": "Encoder-decoder (T5, BART)",
```

```
    "Open-ended text generation": "Decoder-only (GPT family)",
    "Named Entity Recognition": "Encoder-only (BERT, RoBERTa)",
    "Text summarization": "Encoder-decoder (T5, BART)",
}

for task, family in answers.items():
    print(f"{task:30} -> {family}")
```

```
Sentiment classification       -> Encoder-only (BERT, DistilBERT)
Machine translation            -> Encoder-decoder (T5, BART)
Open-ended text generation     -> Decoder-only (GPT family)
Named Entity Recognition       -> Encoder-only (BERT, RoBERTa)
Text summarization             -> Encoder-decoder (T5, BART)
```

Task 2 — Encoder-only Model

Use the sentiment classifier on these three sentences:

```
The movie was excellent.
The lecture was difficult to understand.
I enjoyed learning about transformers.
```

Print the predicted label and score for each sentence.

```
sentences = [
    "The movie was excellent.",
    "The lecture was difficult to understand.",
    "I enjoyed learning about transformers.",
]

for s in sentences:
    r = classifier(s)[0]
    print(f"{s}\n   label: {r['label']}, score: {r['score']:.4f}")
```

```
The movie was excellent.
   label: POSITIVE, score: 0.9999
The lecture was difficult to understand.
   label: NEGATIVE, score: 0.9997
I enjoyed learning about transformers.
   label: POSITIVE, score: 0.9997
```

Task 3 — Encoder–Decoder Model

Use the FLAN-T5 model to perform the following:

- Summarize a short paragraph.

```
paragraph = (
    "Natural language processing is a field of artificial intelligence that focuses on the "
    "interaction between computers and human language. It enables machines to read, understand, "
    "and generate text, and it powers applications such as translation, chatbots, search engines, "
    "and sentiment analysis."
)

prompt = "Summarize: " + paragraph
inputs = tokenizer(prompt, return_tensors="pt")
output = model.generate(**inputs, max_new_tokens=60)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

```
Artificial intelligence is a field of artificial intelligence that focuses on the interaction between computers and human langua
```

Task 5 — Short Reflection

Answer in 3–5 sentences:

Why is a decoder-only model suitable for a chatbot such as ChatGPT?

Answer:

A decoder-only model is trained to predict the next token given all previous tokens, which is exactly what a chatbot does when it writes a reply one token at a time. Its causal (left-to-right) attention lets it condition on the whole conversation so far while generating fluent, open-ended text. The same single architecture handles questions, instructions, and follow-ups just by putting them in the prompt, so no separate encoder is needed. It also scales well with large amounts of unlabeled text, which gives strong general language ability.

Optional Challenge

Try generating text several times with the same prompt.

Remove:

```
set_seed(42)
```

and run the sampling cell multiple times.

Question

Why do you sometimes get different outputs even when the prompt is the same?

Hint: Sampling introduces randomness when choosing the next token.

```
set_seed(42)
sampler = pipeline("text-generation", model="distilgpt2")
outs = sampler("Natural language processing is", max_new_tokens=30, do_sample=True, num_return_sequences=3)
for o in outs:
    print(o["generated_text"])
    print("-----")
```

```
oading weights: 100%                                                    76/76 [00:00<00:00, 849.12it/s]
transformers] Passing `generation_config` together with generation-related arguments=({'do_sample', 'num_return_sequences', 'max
transformers] Both `max_new_tokens` (=30) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. Pl
latural language processing is a form of computation that involves a number of steps.



he process consists of a number of steps.

he process consists of
----
latural language processing is a process that allows you to communicate with the different languages and languages of your langua
```